

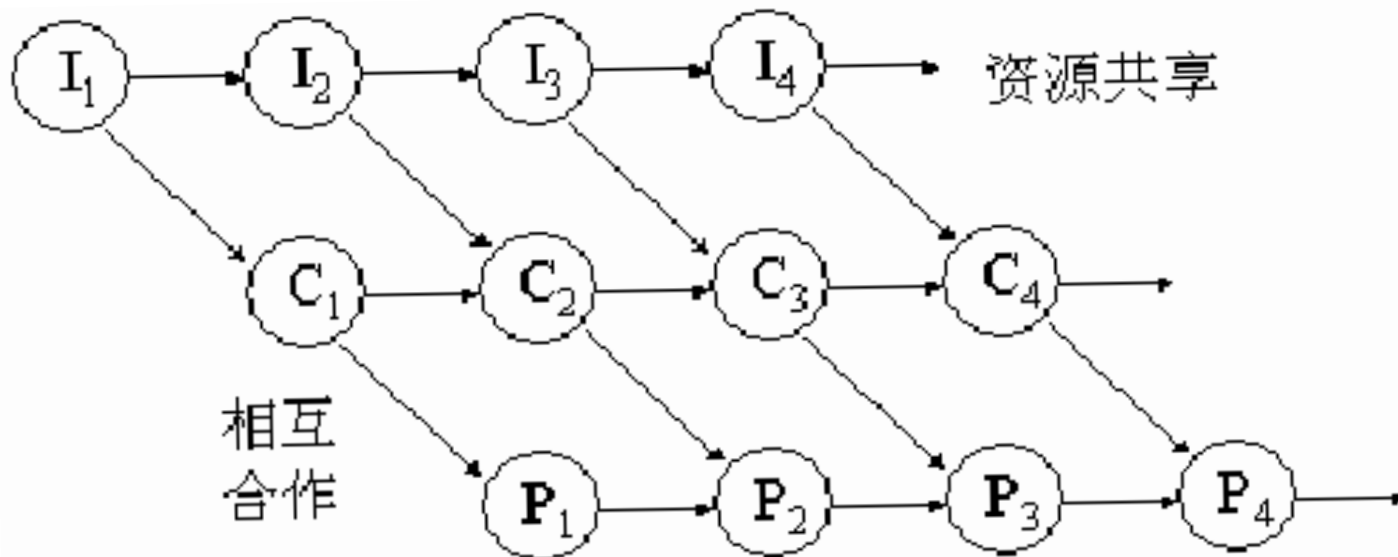
上节回顾

- **进程控制块PCB**
 - 进程标识符、处理机状态
 - 进程调度信息、进程控制信息
- **进程的创建**
 - 申请PCB、分配资源、初始化PCB、插入就绪队列
- **进程的终止**
 - 停止执行、终止子孙进程、回收资源、撤销PCB
- **进程的阻塞：block**
 - 执行过程中主动请求阻塞，释放CPU
 - 保存现场、PCB中设置阻塞状态、插入阻塞队列
- **进程的唤醒：wakeup**
 - 满足资源请求、移出阻塞队列、PCB中设置就绪状态，插入就绪队列
- **进程的挂起\激活：suspend\active**
 - 在活动\静止队列间移动，修改PCB状态、程序数据在内存\硬盘间交换

2.3 进程同步

2.3.1 进程同步的基本概念

- 进程同步的任务：是使并发进行的诸进程之间能有效地共享资源和相互合作，从而使程序的执行具有可再现性. P47
- 两种形式的制约关系
 - 间接相互制约关系：申请I/O、CPU
 - 直接相互制约关系：输入与运算



2.3 进程同步

■ 2. 临界资源：进程间共享的软硬件资源

- 互斥型共享

■ 进程同步的典型例子：生产者-消费者问题

- 生产者进程生产产品，消费者进程消费产品

- 具有 n 个缓冲区的缓冲池

- 生产者进程每次放入一个产品，一个缓冲区满

- 消费者进程每次取走产品，一个缓冲区空

- 不允许生产者进程向满缓冲区放产品

- 不允许消费者进程从空缓冲区取产品

- 互斥共享了缓冲池，每个时刻只有一个进程可以对缓冲池做操作

2.3 进程同步

■ 参数变量

- 数组表示具有 n 个(0, 1, ..., $n-1$)缓冲区的缓冲池
- 输入指针 in 指示下一个空缓冲区，每当生产者进程投放一个产品， $in+1$
- 输出指针 out 指示下一个满缓冲区，每当消费者进程取走一个产品， $out+1$
- 循环缓冲， $in:=(in+1) \bmod n$; $out:=(out+1) \bmod n$
- 当 $(in+1) \bmod n = out$ ，表示缓冲池满
- 当 $in=out$ 表示缓冲池空
- in 和 out 初始化为0
- 整型变量 $counter$ ，初始值为0，记录当前产品数量

2.3 进程同步

producer: repeat

...

produce an item in nextp; //局部变量nextp, 暂时存放产品

...

while counter=n do no-op; //缓冲区全满, 循环等待 ·

buffer [in] := nextp; //产品放入缓冲区

in:= in+1 mod n; //指针移到下一个空缓冲区

counter:= counter+1; //可用产品增加1

until false;

consumer: repeat

while counter=0 do no-op; //缓冲区全空, 循环等待

nextc:=buffer [out] ; //局部变量nextc, 暂存取出的产品

out:=(out+1) mod n; //指针移到下一个满缓冲区

counter:= counter-1; //可用产品减少1

...

■ 共享变量counter

2.3 进程同步

■ counter:= counter+1的指令操作

- register1: = counter;
- register1: = register1+1;
- counter: = register 1;

■ counter:= counter-1的指令操作

- register2: = counter;
- register2: = register2-1;
- counter: = register 2;

2.3 进程同步

■ 并发乱序执行将引发错误

- ❑ **counter**的当前值是5
- ❑ **register 1 : = counter; (register 1=5)**
- ❑ **register 1 : = register 1+1; (register 1=6)**
- ❑ **register 2 : = counter; (register 2=5)**
- ❑ **register 2 : = register 2-1; (register 2=4)**
- ❑ **counter : = register 1; (counter=6)**
- ❑ **counter : = register 2; (counter=4)**
- ❑ 正确执行是**counter+1-1=5**

2.3 进程同步

■ 3. 临界区(critical section)

可把一个访问临界资源的循环进程描述如下： •

repeat

entry section

——进入区，检查

critical section;

——临界区，访问资源

exit section

——退出区，恢复

remainder section;

until false;

2.3 进程同步

■ 4. 同步机制应遵循的规则

- 空闲让进
- 忙则等待
- 有限等待
- 让权等待：释放**CPU**、避免忙等

2.3 进程同步

2.3.2 信号量机制

1. 整型信号量

- 使用一个代表资源数目的整型变量
- 除初始化外，仅两个标准原子操作能访问

□ **wait(S)**和**signal(S)**

□ 分别称为**P**、**V**操作

wait(S): **while** $S \leq 0$ **do no-op**
 S := S-1;

signal(S): **S := S+1;**

- 缺点：**While**循环不能释放处理机，“忙等”，未遵循“让权等待”规则

2.3 进程同步

2. 记录型信号量

- 不存在“忙等”现象，采取“让权等待”的策略
- 增加一个进程链表L，用于链接所有等待进程
- 等待进程使用阻塞操作进入进程链表L
- 信号量S是一个结构体：

type semaphore=record

value:integer; //资源可用数量

L:list of process; //等待进程队列

end

2.3 进程同步

■ PV操作描述

procedure wait(S)

var S: semaphore;

begin

S.value : = S.value-1;

if S.value < 0 then block(S,L) //S.value < 0时，资源已分配完毕，进程调用block原语进行自我阻塞，放弃处理机，并插入到信号量链表S.L中。

end

procedure signal(S)

var S: semaphore;

begin

S.value : = S.value+1;

if S.value ≤ 0 then wakeup(S,L); //若加1后S.value ≤ 0，则表示链表中有等待该资源的进程被阻塞，调用wakeup原语，将S.L链表中的第一个等待进程唤醒

end

2.3 进程同步

- **S.value**的初值为1，表示只允许一个进程访问临界资源，此时的信号量转化为互斥信号量
- **S.value**意义的变化：
 - >0 ，可用资源数量
 - <0 ，阻塞进程个数
 - $=0$ ，临界状态，既无可用资源也无阻塞进程

2.3 进程同步

- 银行存款问题，变量**count**是对应账户余额，对同一个账户两个操作：存款**300**，取款**200**

- ☐ **count**属于共享资源，互斥型访问

- ☐ **P1: count += 300**

- ☐ **P2: count -= 200**

- 无信号量控制的执行顺序

P1: LD R1, count

P2: LD R2, count

P2: SUB R2, 200

P2: LD count, R2

P1: ADD R1, 300

P1: LD count, R1

结果: count + 300

P1: LD R1, count

P2: LD R2, count

P1: ADD R1, 300

P1: LD count, R1

P2: SUB R2, 200

P2: LD count, R2

结果: count - 200

2.3 进程同步

- 加入信号量控制的存取款程序

var mutex: semaphore := 1;

Procedure P1 (m) //存款程序

being

wait(mutex);

count += m;

signal(mutex);

end

//信号量定义

Procedure P2 (n) //取款程序

being

wait(mutex);

count -= n;

signal(mutex);

end

- 存取款操作无论执行哪个先后都能保证正确性

□ P1(300) → P2(200)

□ P2(200) → P1(300)

2.3 进程同步

3. AND型信号量

■ 引发死锁的情况

- 两个进程包含对两个信号量**Dmutex**和**Emutex**的操作

process A:

wait(Dmutex);

wait(Emutex);

process B:

wait(Emutex);

wait(Dmutex);

- 引发死锁的操作次序（两个信号量都初始为1）

process A: wait(Dmutex); //这时**Dmutex=0**

process B: wait(Emutex); //这时**Emutex=0**

process A: wait(Emutex); //这时**Emutex=-1**，**A阻塞**

process B: wait(Dmutex); //这时**Dmutex=-1**，**B阻塞**

2.3 进程同步

- **AND**同步机制的基本思想是：将进程在整个运行过程中需要的所有资源，一次性全部地分配给进程，待进程使用完后再一起释放。只要尚有一个资源未能分配给进程，其它所有可能为之分配的资源，也不分配给他。
- 对若干个临界资源的分配，采取原子操作方式：要么全部分配到进程，要么一个也不分配。
- 由死锁理论可知，这样就可避免上述死锁情况的发生。为此，在**wait**操作中，增加了一个“**AND**”条件，故称为**AND**同步，或称为同时**wait**操作。包含操作**Swait**和**Ssignal**

2.3 进程同步

■ **Swait**操作定义:

```
Swait(S1, S2, ..., Sn)           //n种资源
  if  $S_i \geq 1$  and ... and  $S_n \geq 1$  then //每种资源数量都大于1才分配
    for i := 1 to n do
       $S_i := S_i - 1$ ;
    endfor
  else //不能满足资源分配
    将进程插入首个令它阻塞的资源阻塞队列，并将执行指针
    移动回到程序Swait开始位置，程序被唤醒后需要重新进行资源判断
  endif
```

2.3 进程同步

■ Ssignal操作定义:

Ssignal(S1, S2, ..., Sn)

for i: = 1 to n do //每种被占用的资源都释放

Si=Si+1;

将每种资源的阻塞队列中的所有进程移出，唤醒重新判断

endfor;

2.3 进程同步

- 记录型信号量：请求1种资源，数量1个
- **AND**型信号量：请求n种资源，每种数量1个
- 信号量集：请求n种资源，每种数量n个
- 信号量**S**，需求值**d**，下限值**t**，操作**Swait**和**Ssignal**

Swait(S1, t1, d1, ..., Sn, tn, dn)

if $S_i \geq t_1$ and ... and $S_n \geq t_n$ then //所有资源种类大于下限值才分配

for i:=1 to n do

$S_i := S_i - d_i;$ //分配资源

endfor

else

将进程插入首个令它阻塞的资源阻塞队列，并将执行指针移动回到程序**Swait**开始位置，程序被唤醒后需要重新进行资源判断。

endif

2.3 进程同步

■ Ssignal定义

Ssignal(S1, d1, ..., Sn, dn)

for i := 1 to n do

$S_i := S_i + d_i$; //释放资源

 将每种资源的阻塞队列中的所有进程移出，唤醒重新判断

endfor;

■ 信号量集的几种特殊情况：

- **Swait(S, d, d)**: 此时在信号量集中只有一个信号量**S**，但允许它每次申请**d**个资源，当现有资源数少于**d**时，不予分配。·
- **Swait(S, 1, 1)**: 信号量集蜕化为一般的记录型信号量(**S**>1时)或互斥信号量(**S**=1时)
- **Swait(S, 1, 0)**: 这是一种很特殊且很有用的信号量操作。当**S**≥1时，允许多个进程进入某特定区；当**S**变为0后，将阻止任何进程进入特定区。它相当于一个可控开关。